



Queries with aggregates

In addition to the simple expressions that we introduced last lesson, SQL also supports the use of aggregate expressions (or functions) that allow you to summarize information about a group of rows of data. With the Pixar database that you've been using, aggregate functions can be used to answer questions like, "How many movies has Pixar produced?", or "What is the highest grossing Pixar film each year?".

Select query with aggregate functions over all rows

```
SELECT AGG_FUNC(column_or_expression) AS aggregate_description, ...
FROM mytable
WHERE constraint_expression;
```

Without a specified grouping, each aggregate function is going to run on the whole set of result rows and return a single value. And like normal expressions, giving your aggregate functions an alias ensures that the results will be easier to read and process.

Common aggregate functions

Here are some common aggregate functions that we are going to use in our examples:

Function	Description
COUNT(*), COUNT(<i>column</i>)	A common function used to counts the number of rows in the group if no column name is specified. Otherwise, count the number of rows in the group with non-NULL values in the specified column.
MIN(<i>column</i>)	Finds the smallest numerical value in the specified column for all rows in the group.
MAX(<i>column</i>)	Finds the largest numerical value in the specified column for all rows in the group.
AVG(<i>column</i>)	Finds the average numerical value in the specified column for all rows in the group.
SUM(<i>column</i>)	Finds the sum of all numerical values in the specified column for the rows in the group.

Grouped aggregate functions

In addition to aggregating across all the rows, you can instead apply the aggregate functions to individual groups of data within that group (ie. box office sales for Comedies vs Action movies). This would then create as many results as there are unique groups defined as by the **GROUP BY** clause.

Select query with aggregate functions over groups

```
SELECT AGG_FUNC(column_or_expression) AS aggregate_description, ...
FROM mytable
WHERE constraint_expression
GROUP BY column;
```

The **GROUP BY** clause works by grouping rows that have the same value in the column specified.



Exercise

For this exercise, we are going to work with our **Employees** table. Notice how the rows in this table have shared data, which will give us an opportunity to use aggregate functions to summarize some high-level metrics about the teams. Go ahead and give it a shot.

Exercise – Tasks

1. Find the longest time that an employee has been at the studio
2. For each role, find the average number of years employed by employees in that role
3. Find the total number of employee years worked in each building

Our queries are getting fairly complex, but we have nearly introduced all the important parts of a **SELECT** query. One thing that you might have noticed is that if the **GROUP BY** clause is executed after the **WHERE** clause (which filters the rows which are to be grouped), then how exactly do we filter the grouped rows?

Luckily, SQL allows us to do this by adding an additional **HAVING** clause which is used specifically with the **GROUP BY** clause to allow us to filter grouped rows from the result set.

Select query with HAVING constraint

```
SELECT group_by_column, AGG_FUNC(column_expression) AS aggregate_result_alias, ...  
FROM mytable  
WHERE condition  
GROUP BY column  
HAVING group_condition;
```

The **HAVING** clause constraints are written the same way as the **WHERE** clause constraints, and are applied to the grouped rows. With our examples, this might not seem like a particularly useful construct, but if you imagine data with millions of rows with different properties, being able to apply additional constraints is often necessary to quickly make sense of the data.

Did you know?

If you aren't using the ``GROUP BY`` clause, a simple ``WHERE`` clause will suffice.

Exercise

For this exercise, you are going to dive deeper into **Employee** data at the film studio. Think about the different clauses you want to apply for each task.

Exercise – Tasks

1. Find the number of Artists in the studio (without a **HAVING** clause)
2. Find the number of Employees of each role in the studio
3. Find the total number of years employed by all Engineers